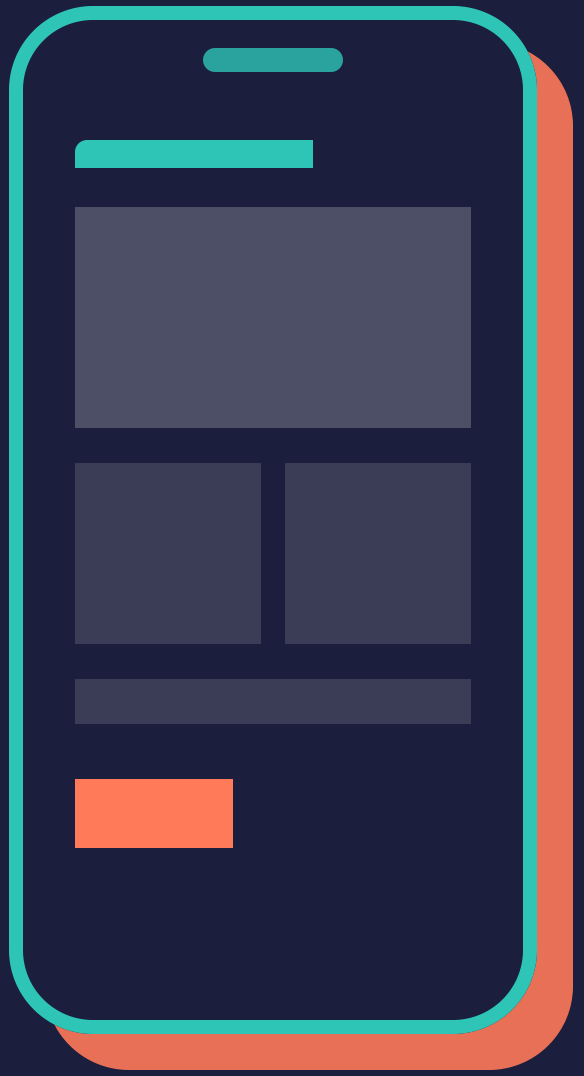
Navigation et listes

Cours 3 – Expo Router, données distantes et listes

Chris CHEVALIER



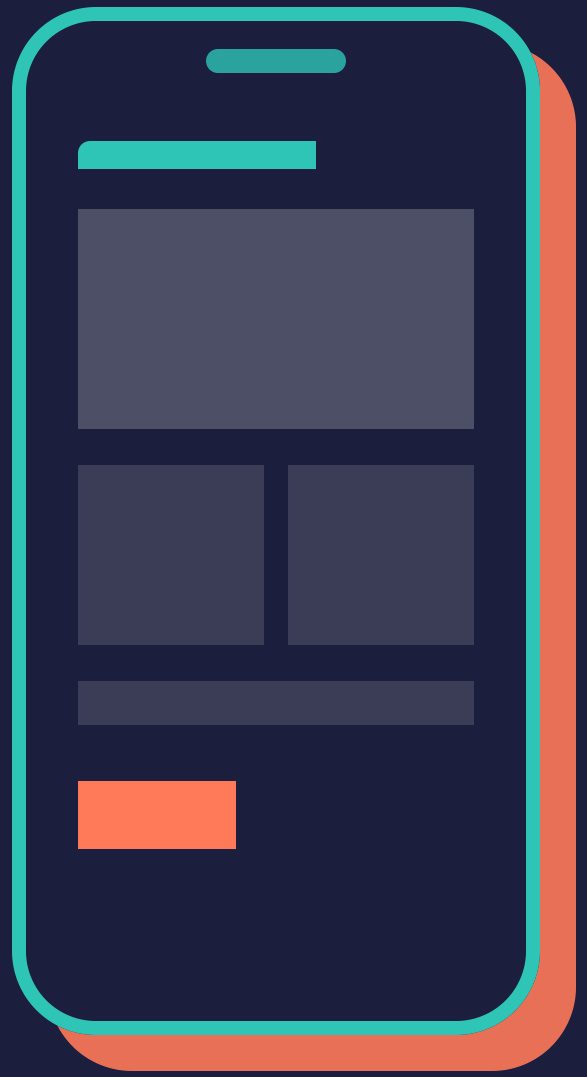
Sommaire

01 Naviguer sur mobile et sur le web

02 Expo Router : les fichiers sont les routes

03 Onglets, pile et routes dynamiques

04 Les listes volumineuses



01

Naviguer sur mobile et sur le web

Deux logiques à réconcilier

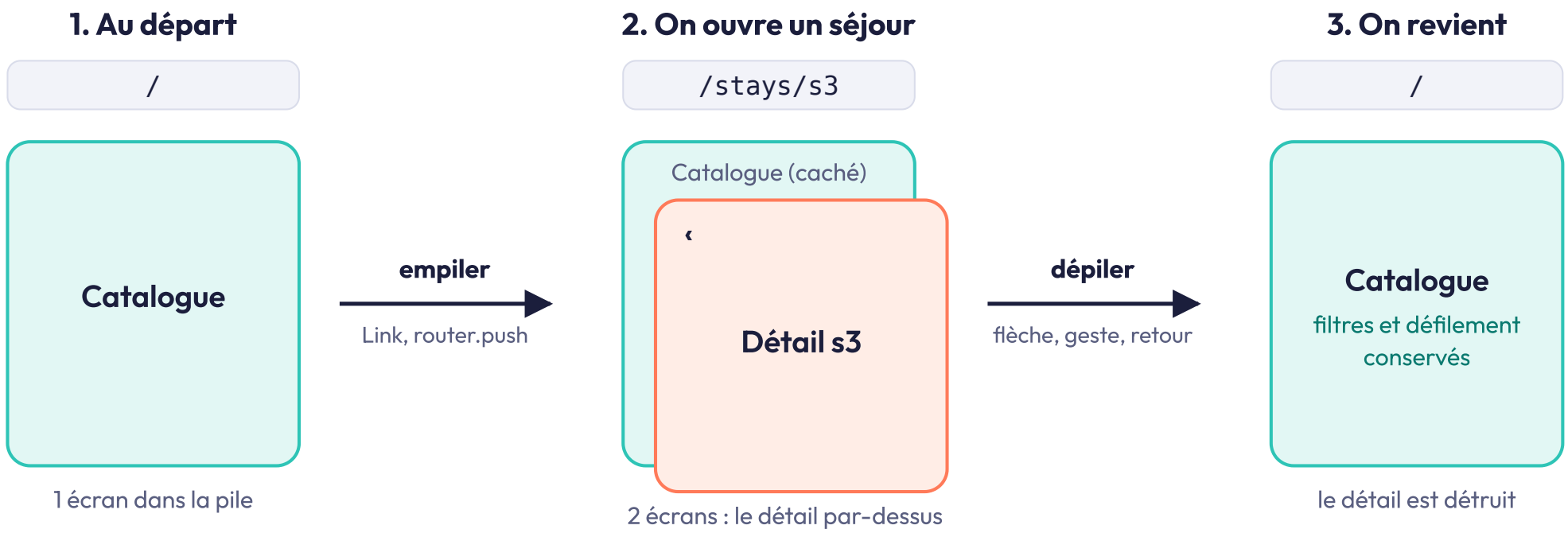


Deux façons de naviguer

	Sur le web	Sur mobile
Ce qui définit l'écran	L'URL dans la barre d'adresse	Une **pile** d'écrans en mémoire
Le retour	Le bouton du navigateur	La flèche de l'en-tête, le geste ou le bouton système
Les sections	Un menu	Des **onglets** en bas de l'écran
Ouvrir un contenu de l'extérieur	Un lien partagé	Un **lien profond** (deep link) qui ouvre l'application sur le bon écran

Avec **Expo Router**, chaque écran a une URL. La même navigation produit une pile sur mobile et des adresses dans le navigateur.

La pile d'écrans



> Expo Router s'appuie sur React Navigation, la bibliothèque de navigation de référence. Vous n'avez pas à la configurer vous-mêmes.

02

Expo Router : les fichiers sont les routes

Le routage par fichiers



Un fichier, une URL

src/app/

```
src/app/
├── _layout.tsx          pile racine
├── +not-found.tsx       URL inconnue
├── (tabs)/
│   ├── _layout.tsx      onglets
│   ├── index.tsx        /
│   └── favorites.tsx    /favorites
└── stays/
    └── [id].tsx         /stays/s1, /stays/s2…
```

- Un nom ordinaire devient un morceau d'URL. `_` : le fichier n'est **pas une route**. `_layout.tsx` est le navigateur de son dossier
- `( )` : un **groupe**, absent de l'URL. `(tabs)/favorites.tsx` répond à `/favorites`
- `[ ]` : un **segment dynamique**, récupéré comme paramètre (`id` vaut `s3`)
- `+` : une **route spéciale** d'Expo Router. `+not-found` répond aux adresses inconnues

Préparer le projet

Terminal

```
npx expo install @expo/vector-icons
mkdir "src/app/(tabs)"
mv src/app/index.tsx "src/app/(tabs)/"
```

- Les icônes des onglets viennent de `@expo/vector-icons`. `npx expo install` choisit la version compatible avec le SDK
- Le catalogue devient le premier onglet : on le range dans le groupe `(tabs)`

03

Onglets, pile et routes dynamiques

Construire le parcours de l'utilisateur



Le deuxième onglet : les favoris

src/app/(tabs)/favorites.tsx

```tsx
import { FlatList, StyleSheet, Text } from "react-native";

import { StateMessage } from "@/components/state-message";
import { useFavorites } from "@/hooks/use-favorites";
import { useStays } from "@/hooks/use-stays";

export default function FavoritesScreen() {
  const { state } = useStays();
  const { isFavorite } = useFavorites();

  if (state.status !== "success")
    return <StateMessage loading={state.status === "loading"} />;
  const favoriteStays = state.stays.filter((stay) =>
    isFavorite(stay.id),
  );

  return (
    <FlatList
      data={favoriteStays}
      keyExtractor={(stay) => stay.id}
```

- Un nouvel écran dans `src/app/(tabs)/` : il s'ouvre à l'adresse `/favorites`
- Il lit les favoris dans le Context, comme le badge du catalogue
- La liste affichée se **calcule** à partir des séjours et des favoris : aucun état en double

Le deuxième onglet : les favoris (suite)

src/app/(tabs)/favorites.tsx

```tsx
      renderItem={({ item }) => (
        <Text style={styles.item}>
          {item.title} · {item.city}
        </Text>
      )}
      ListEmptyComponent={
        <Text style={styles.item}>Aucun favori pour le moment.</Text>
      }
      contentContainerStyle={styles.list}
    />
  );
}

const styles = StyleSheet.create({
  list: { padding: 16, gap: 8 },
  item: { fontSize: 17 },
});
```

Les onglets

src/app/(tabs)/_layout.tsx

```tsx
import Ionicons from "@expo/vector-icons/Ionicons";
import { Tabs } from "expo-router/js-tabs";

const ACTIVE = "#0B7A70";
const INACTIVE = "#8C90B4";

export default function TabsLayout() {
  return (
    <Tabs
      screenOptions={{
        tabBarActiveTintColor: ACTIVE,
        tabBarInactiveTintColor: INACTIVE,
      }}
    >
      <Tabs.Screen
        name="index"
        options={{
          title: "Séjours",
          tabBarIcon: ({ focused, size }) => (
            <Ionicons
```

- `name` reprend le nom du fichier
- Deux en-têtes s'empilent : celui de la pile et celui des onglets. La diapo suivante masque celui de la pile
- Icônes : `@expo/vector-icons`, à installer avec `npx expo install`

Les onglets (suite)

src/app/(tabs)/_layout.tsx

```
            name="bed-outline"
            size={size}
            color={focused ? ACTIVE : INACTIVE}
          />
        ),
      }}
    />
    <Tabs.Screen
      name="favorites"
      options={{
        title: "Favoris",
        tabBarIcon: ({ focused, size }) => (
          <Ionicons
            name="heart-outline"
            size={size}
            color={focused ? ACTIVE : INACTIVE}
          />
        ),
      }}
    />
```

Les onglets (suite)

src/app/(tabs)/_layout.tsx

```
41      </Tabs>
42    );
43  }
```

La pile racine

src/app/_layout.tsx · nouveau code surligné

```tsx
import { Stack } from "expo-router";

import { FavoritesProvider } from "@/context/favorites-context";

// Un séjour ouvert directement par son URL (lien partagé) est posé
// sur les onglets : sa flèche de retour y mène
export const unstable_settings = { anchor: "(tabs)" };

export default function RootLayout() {
  return (
    <FavoritesProvider>
      <Stack screenOptions={{ headerTransparent: false }}>
        <Stack.Screen
          name="(tabs)"
          options={{ headerShown: false, title: "Séjours" }}
        />
      </Stack>
    </FavoritesProvider>
  );
}
```

- Les onglets forment **un seul écran** de la pile. Son en-tête est masqué car les onglets ont le leur
- Son `title` s'affiche dans le bouton retour du détail, à la place de « (tabs) »
- `anchor` : l'écran placé **sous** un séjour ouvert directement par son URL (un lien partagé). Sans lui, pas de flèche de retour. `unstable_` signale une option qui peut encore changer

Une page introuvable : +not-found

src/app/+not-found.tsx

```tsx
import { Link, Stack } from "expo-router";
import { StyleSheet, Text, View } from "react-native";

export default function NotFoundScreen() {
  return (
    <View style={styles.box}>
      <Stack.Screen options={{ title: "Page introuvable" }} />
      <Text style={styles.text}>Cette page est introuvable.</Text>
      <Link href="/" style={styles.link}>
        Retour aux séjours
      </Link>
    </View>
  );
}

const styles = StyleSheet.create({
  box: {
    flex: 1,
    alignItems: "center",
    justifyContent: "center",
```

- Expo Router affiche cet écran pour toute adresse inconnue
- `<Link href="/">` ramène à l'accueil
- Testez sur le web avec une adresse inventée, par exemple `/vacances`

Une page introuvable : +not-found (suite)

src/app/+not-found.tsx

```
    gap: 12,
  },
  text: { fontSize: 18 },
  link: { fontSize: 16, color: "#0B7A70", fontWeight: "600" },
});
```

Charger un seul séjour

src/services/stays-service.ts

```ts
import { Stay, StayType } from "@/models/stay";

const SITE = "https://chris-chevalier.github.io/react-native-supports";
const STAYS_URL = `${SITE}/api/stays.json`;

// Forme de la réponse de l'API, limitée à ce qu'on utilise
type StayResponse = {
  id: string;
  title: string;
  type: StayType;
  price_per_night: number;
  is_available: boolean;
  image_url: string | null;
  location: { city: string; latitude: number; longitude: number };
};

// Traduit une réponse de l'API dans le vocabulaire du modèle
function toStay(response: StayResponse): Stay {
  return {
    id: response.id,
```

- **Seule la dernière fonction, `fetchStay`, est nouvelle** : elle est surlignée. Le reste du fichier existe déjà
- Le détail ne reçoit qu'un identifiant : le service retrouve le séjour
- Il renvoie `undefined` si l'identifiant est inconnu. L'écran devra le prévoir

Charger un seul séjour (suite)

src/services/stays-service.ts · nouveau code surligné

```
    title: response.title,
    city: response.location.city,
    type: response.type,
    pricePerNight: response.price_per_night,
    available: response.is_available,
    latitude: response.location.latitude,
    longitude: response.location.longitude,
    imageUrl: response.image_url ?? undefined,
  };
}

export async function fetchStays(): Promise<Stay[]> {
  const response = await fetch(STAYS_URL);
  if (!response.ok) throw new Error(`Erreur HTTP ${response.status}`);
  const data: StayResponse[] = await response.json();
  return data.map(toStay);
}

export async function fetchStay(
  id: string,
```

Charger un seul séjour (suite)

src/services/stays-service.ts · nouveau code surligné

```
41 ): Promise<Stay | undefined> {
42   const stays = await fetchStays();
43   return stays.find((stay) => stay.id === id);
44 }
```

Le hook useStay

src/hooks/use-stay.ts

```ts
import { useEffect, useState } from "react";

import { Stay } from "@/models/stay";
import { fetchStay } from "@/services/stays-service";

export type StayState =
  | { status: "loading" }
  | { status: "not-found" }
  | { status: "success"; stay: Stay };

export function useStay(id: string) {
  const [state, setState] = useState<StayState>({
    status: "loading",
  });

  useEffect(() => {
    let cancelled = false;
    fetchStay(id).then((stay) => {
      if (!cancelled)
        setState(
```

- Trois états : chargement, introuvable, succès
- `id` est une dépendance de l'effet : changer de séjour recharge les données

Le hook useStay (suite)

src/hooks/use-stay.ts

```ts
          stay
            ? { status: "success", stay }
            : { status: "not-found" },
        );
    });
    return () => {
      cancelled = true;
    };
  }, [id]);

  return state;
}
```

Une route dynamique

src/app/stays/[id].tsx

```tsx
import { Stack, useLocalSearchParams } from "expo-router";
import {
  Image,
  ScrollView,
  StyleSheet,
  Text,
  View,
} from "react-native";

import { StateMessage } from "@/components/state-message";
import { useStay } from "@/hooks/use-stay";
import { Stay } from "@/models/stay";

export default function StayScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const state = useStay(id);

  if (state.status === "loading") return <StateMessage loading />;
  if (state.status === "not-found")
    return <StateMessage message="Ce séjour est introuvable." />;
```

- `useLocalSearchParams` lit `id` dans l'URL
- L'écran ne reçoit **que l'identifiant**, jamais l'objet entier : il recharge ses données. C'est ce qui permet d'ouvrir la page depuis un lien
- `<Stack.Screen options>` change le titre de l'en-tête
- Un identifiant inconnu est un cas prévu

Une route dynamique (suite)

src/app/stays/[id].tsx

```tsx
  return <StayDetails stay={state.stay} />;
}

function StayDetails({ stay }: { stay: Stay }) {
  return (
    <ScrollView contentContainerStyle={styles.content}>
      <Stack.Screen options={{ title: stay.title }} />
      <Image source={{ uri: stay.imageUrl }} style={styles.image} />
      <View style={styles.body}>
        <Text style={styles.title}>{stay.title}</Text>
        <Text style={styles.city}>
          {stay.city} · {stay.pricePerNight} € / nuit
        </Text>
      </View>
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  content: { width: "100%", maxWidth: 800, alignSelf: "center" },
```

Une route dynamique (suite)

src/app/stays/[id].tsx

```
41    image: { width: "100%", aspectRatio: 16 / 9 },
42    body: { padding: 16, gap: 8 },
43    title: { fontSize: 26, fontWeight: "800" },
44    city: { fontSize: 17, color: "#5A5F80" },
45  });
```

URL et liens profonds

Sur le web

- L'URL `/stays/s3` s'affiche dans la barre d'adresse
- On peut la copier, la partager, la recharger : le même écran s'ouvre
- Le bouton retour du navigateur dépile

Sur mobile

- Le `scheme` d' `app.json` définit le préfixe : `staybnb://stays/s3`
- Dans Expo Go, l'adresse devient `exp://<IP>:8081/--/stays/s3`
- Une notification ou un e-mail peut ouvrir l'application sur le bon séjour

Naviguer avec un lien : `Link`

src/app/(tabs)/favorites.tsx · nouveau code surligné

```tsx
import { Link } from "expo-router";
import { FlatList, StyleSheet, Text } from "react-native";

import { StateMessage } from "@/components/state-message";
import { useFavorites } from "@/hooks/use-favorites";
import { useStays } from "@/hooks/use-stays";

export default function FavoritesScreen() {
  const { state } = useStays();
  const { isFavorite } = useFavorites();

  if (state.status !== "success")
    return <StateMessage loading={state.status === "loading"} />;
  const favoriteStays = state.stays.filter((stay) =>
    isFavorite(stay.id),
  );

  return (
    <FlatList
      data={favoriteStays}
```

- `<Link>` est l'équivalent de `<a href>` : sur le web, c'est un vrai lien
- `href` reçoit le chemin de la route et ses paramètres : ici l'identifiant du séjour
- Les **routes typées** (`typedRoutes` dans `app.json`) font signaler par TypeScript un lien vers une page inexistante

Naviguer avec un lien : Link (suite)

src/app/(tabs)/favorites.tsx · nouveau code surligné

```
21        keyExtractor={(stay) => stay.id}
22        renderItem={({ item }) => (
23          <Link
24            href={{ pathname: "/stays/[id]", params: { id: item.id } }}
25            style={styles.item}
26          >
27            {item.title}
28          </Link>
29        )}
30        ListEmptyComponent={
31          <Text style={styles.item}>Aucun favori pour le moment.</Text>
32        }
33        contentContainerStyle={styles.list}
34      />
35    );
36  }
37
38  const styles = StyleSheet.create({
39    list: { padding: 16 },
40    item: { fontSize: 17, paddingVertical: 8 },
```

Naviguer avec un lien : `Link` (suite)

src/app/(tabs)/favorites.tsx

```
41 });
```

Une carte qui s'ouvre au toucher

src/components/stay-card.tsx · nouveau code surligné

```tsx
import {
  Image,
  Pressable,
  StyleSheet,
  Text,
  View,
} from "react-native";

import { Stay } from "@/models/stay";

type StayCardProps = {
  stay: Stay;
  isFavorite: boolean;
  onToggleFavorite: (id: string) => void;
  onPress?: (id: string) => void;
};

const formatPrice = (price: number) =>
  new Intl.NumberFormat("fr-FR", {
    style: "currency",
```

- Toute la carte devient une zone d'appui. Le bouton favori garde sa propre action
- `onPress` est facultative : une carte sans action reste utilisable ailleurs
- `onPress?.(stay.id)` n'appelle la fonction que si elle existe

Une carte qui s'ouvre au toucher (suite)

src/components/stay-card.tsx · nouveau code surligné

```
    currency: "EUR",
    maximumFractionDigits: 0,
  }).format(price);

export function StayCard({
  stay,
  isFavorite,
  onToggleFavorite,
  onPress,
}: StayCardProps) {
  return (
    <Pressable onPress={() => onPress?.(stay.id)} style={styles.card}>
      <Image source={{ uri: stay.imageUrl }} style={styles.image} />
      <View style={styles.body}>
        <Text style={styles.title}>{stay.title}</Text>
        <Text style={styles.city}>{stay.city}</Text>
        <Text style={styles.price}>
          {formatPrice(stay.pricePerNight)} / nuit
        </Text>
        {!stay.available && (
```

Une carte qui s'ouvre au toucher (suite)

src/components/stay-card.tsx · nouveau code surligné

```
41            <Text style={styles.unavailable}>Complet</Text>
42          )}
43          <Pressable
44            onPress={() => onToggleFavorite(stay.id)}
45            style={({ pressed }) => [
46              styles.button,
47              pressed && styles.buttonPressed,
48            ]}
49          >
50            <Text style={styles.buttonLabel}>
51              {isFavorite ? "★ Favori" : "☆ Ajouter aux favoris"}
52            </Text>
53          </Pressable>
54        </View>
55      </Pressable>
56    );
57  }
58
59  const styles = StyleSheet.create({
60    card: {
```

Une carte qui s'ouvre au toucher (suite)

src/components/stay-card.tsx

```
    flex: 1,
    backgroundColor: "#fff",
    borderRadius: 12,
    overflow: "hidden",
    margin: 8,
  },
  image: { width: "100%", aspectRatio: 4 / 3 },
  body: { padding: 12, gap: 4 },
  title: { fontSize: 18, fontWeight: "700" },
  city: { fontSize: 15, color: "#5A5F80" },
  price: { fontSize: 16, color: "#0B7A70" },
  unavailable: { fontSize: 14, color: "#B42318" },
  button: {
    marginTop: 8,
    paddingVertical: 10,
    borderRadius: 8,
    backgroundColor: "#E2F7F4",
    alignItems: "center",
  },
  buttonPressed: { opacity: 0.6 },
```

Une carte qui s'ouvre au toucher (suite)

src/components/stay-card.tsx

```
81    buttonLabel: { fontSize: 15, fontWeight: "600" },
82  });
```

Naviguer par le code : `useRouter`

src/app/(tabs)/index.tsx · nouveau code surligné

```tsx
import { useRouter } from "expo-router";
import { FlatList, StyleSheet, Text, View } from "react-native";

import { StateMessage } from "@/components/state-message";
import { StayCard } from "@/components/stay-card";
import { StayFiltersBar } from "@/components/stay-filters-bar";
import { useFavorites } from "@/hooks/use-favorites";
import { useColumns } from "@/hooks/use-columns";
import { useStayFilters } from "@/hooks/use-stay-filters";
import { useStays } from "@/hooks/use-stays";

export default function CatalogScreen() {
  const router = useRouter();
  const columns = useColumns();
  const { state, reload } = useStays();
  const stays = state.status === "success" ? state.stays : [];
  const { filters, updateFilters, visibleStays } =
    useStayFilters(stays);
  const { isFavorite, toggleFavorite } = useFavorites();
```

- Pour naviguer après une action (un appui, un envoi de formulaire), sans lien visible : `useRouter()` renvoie l'objet `router`

Méthode	Effet
`push(href)`	Empile un écran
`back()`	Revient à l'écran précédent
`replace(href)`	Remplace l'écran actuel (après une connexion, par exemple)
`navigate(href)`	Va à l'écran, sans l'empiler s'il existe déjà

Naviguer par le code : useRouter (suite)

src/app/(tabs)/index.tsx · nouveau code surligné

```tsx
21  const openStay = (id: string) =>
22    router.push({ pathname: "/stays/[id]", params: { id } });
23
24  if (state.status === "loading") return <StateMessage loading />;
25  if (state.status === "error")
26    return <StateMessage message={state.message} onRetry={reload} />;
27
28  return (
29    <View style={styles.container}>
30      <StayFiltersBar filters={filters} onChange={updateFilters} />
31      <FlatList
32        key={columns}
33        numColumns={columns}
34        data={visibleStays}
35        keyExtractor={(stay) => stay.id}
36        renderItem={({ item }) => (
37          <StayCard
38            stay={item}
39            isFavorite={isFavorite(item.id)}
40            onToggleFavorite={toggleFavorite}
```

Naviguer par le code : `useRouter` (suite)

src/app/(tabs)/index.tsx · nouveau code surligné

```
41              onPress={openStay}
42            />
43          )}
44          ListEmptyComponent={
45            <Text style={styles.empty}>
46              Aucun logement ne correspond.
47            </Text>
48          }
49        />
50      </View>
51    );
52  }
53
54  const styles = StyleSheet.create({
55    container: {
56      flex: 1,
57      width: "100%",
58      maxWidth: 1100,
59      alignSelf: "center",
60      padding: 8,
```

Naviguer par le code : `useRouter` (suite)

src/app/(tabs)/index.tsx

```
61    },
62    empty: { margin: 8, fontSize: 16 },
63  });
```

Quiz

Quelle URL correspond au fichier `src/app/(tabs)/favorites.tsx` ?

> `/favorites` : les groupes entre parenthèses n'apparaissent pas dans l'URL.

Pourquoi passer l'identifiant du séjour au détail plutôt que l'objet complet ?

> L'URL doit suffire à afficher la page : ouverte depuis un lien ou rechargée, elle n'a que l'identifiant. Le détail recharge donc ses données.

04

Les listes volumineuses

FlatList et SectionList



`SectionList` : regrouper dans le modèle

src/models/stay.ts

```ts
export type StayType = "apartment" | "house" | "cabin";

export type Stay = {
  id: string;
  title: string;
  city: string;
  type: StayType;
  pricePerNight: number;
  available: boolean;
  latitude: number;
  longitude: number;
  imageUrl?: string;
};

export type StayFilters = {
  query: string;
  type: StayType | null;
  onlyAvailable: boolean;
};
```

- Chaque section a un `title` et un tableau `data`
- Le regroupement est une **règle du modèle**, pure et testable

`SectionList` : regrouper dans le modèle (suite)

src/models/stay.ts

```ts
export const STAY_TYPE_LABELS: Record<StayType, string> = {
  apartment: "Appartement",
  house: "Maison",
  cabin: "Cabane",
};

// Règle métier : une fonction pure, sans React, facile à tester
export function matchesFilters(
  stay: Stay,
  filters: StayFilters,
 ): boolean {
  const search = filters.query.trim().toLowerCase();
  const matchesQuery =
    stay.title.toLowerCase().includes(search) ||
    stay.city.toLowerCase().includes(search);
  const matchesType =
    filters.type === null || stay.type === filters.type;
  const matchesAvailability =
    !filters.onlyAvailable || stay.available;
  return matchesQuery && matchesType && matchesAvailability;
```

`SectionList` : regrouper dans le modèle (suite)

src/models/stay.ts · nouveau code surligné

```ts
}

export type CitySection = { title: string; data: Stay[] };

// Regroupe des séjours par ville, au format attendu par SectionList
export function groupByCity(stays: Stay[]): CitySection[] {
  const cities = [...new Set(stays.map((stay) => stay.city))].sort();
  return cities.map((city) => ({
    title: city,
    data: stays.filter((stay) => stay.city === city),
  }));
}
```

`SectionList` : afficher les groupes

src/app/(tabs)/favorites.tsx · nouveau code surligné

```tsx
import { Link } from "expo-router";
import { SectionList, StyleSheet, Text } from "react-native";

import { StateMessage } from "@/components/state-message";
import { useFavorites } from "@/hooks/use-favorites";
import { useStays } from "@/hooks/use-stays";
import { groupByCity } from "@/models/stay";

export default function FavoritesScreen() {
  const { state } = useStays();
  const { isFavorite } = useFavorites();

  if (state.status !== "success")
    return <StateMessage loading={state.status === "loading"} />;
  const sections = groupByCity(
    state.stays.filter((stay) => isFavorite(stay.id)),
  );

  return (
    <SectionList
```

- `SectionList` est un composant de base, comme `FlatList` : `sections` remplace son `data`
- `renderSectionHeader` affiche le titre de chaque groupe, `renderItem` chaque élément
- Une liste longue reste fluide si chaque élément a une clé stable (`keyExtractor`) et si le tri est fait **avant**, dans le modèle ou le hook. Au-delà de quelques milliers de lignes : FlashList

`SectionList` : afficher les groupes (suite)

src/app/(tabs)/favorites.tsx · nouveau code surligné

```
21        sections={sections}
22        keyExtractor={(stay) => stay.id}
23        renderSectionHeader={({ section }) => (
24          <Text style={styles.city}>{section.title}</Text>
25        )}
26        renderItem={({ item }) => (
27          <Link
28            href={{ pathname: "/stays/[id]", params: { id: item.id } }}
29            style={styles.item}
30          >
31            {item.title}
32          </Link>
33        )}
34        ListEmptyComponent={
35          <Text style={styles.item}>Aucun favori pour le moment.</Text>
36        }
37        contentContainerStyle={styles.list}
38      />
39    );
40  }
```

`SectionList` : afficher les groupes (suite)

src/app/(tabs)/favorites.tsx · nouveau code surligné

```
41
42 const styles = StyleSheet.create({
43   list: { padding: 16 },
44   city: {
45     fontSize: 14,
46     fontWeight: "700",
47     textTransform: "uppercase",
48     color: "#0B7A70",
49     marginTop: 16,
50   },
51   item: { fontSize: 17, paddingVertical: 8 },
52 });
```

À retenir

- Avec Expo Router, **chaque fichier de `src/app/` est une route** et chaque écran a une URL
- `_layout.tsx` choisit le navigateur : pile (`Stack`) ou onglets (`Tabs` depuis `expo-router/js-tabs`)
- Une route dynamique reçoit un identifiant et **recharge** ses données
- Un service appelle l'API avec `fetch`, teste `response.ok` et traduit la réponse en modèle
- Des listes légères, filtrées en amont. `SectionList` pour des groupes

Des questions ?

Chris CHEVALIER
chevalier@chris-freelance.com

